

```
ssize_t sending(int sd, const struct msghdr *message, int flags);
ssize_t receiving(int sd, struct msghdr *message, int flags);
```

Note that *sd* is the socket descriptor of the endpoint, and *message* is the pointer to the *msghdr* structure that contains the user message and ancillary data in the format:

```
struct msghdr {
    void *msg_name;
    socklen_t msg_namelen;
    struct iovec *msg_iov;
    size_t msg_iovlen;
    void *msg_control;
    socklen_t msg_controllen;
    int msg_flags;
};
```

...where:

- *msg_name* specifies the pointer to the socket address structure.
- *msg_namelen* specifies the size of the socket address structure.
- *msg_iov* includes an array of message buffers.
- *msg_iovlen* specifies the number of elements in the *msg_iov* structure.
- *msg_control* specifies the ancillary data.
- *msg_controllen* specifies the length of the ancillary data buffer.
- *msg_flags* specifies the flags on the received message.

Setting socket options

Socket options can be set using the *setsockopt* call—that is, options like enabling/disabling events (SCTP_EVENTS), RTO info (SCTP_RTOINFO), the number of in/out streams (SCTP_INITMSG), etc. For example, SCTP events can be enabled in the following manner:

```
struct sctp_event_subscribe events; /* Structure defined in net/sctp/user.h */
events.sctp_data_in_event = 1; /* 1 == interested in the event, 0 == not
interested */
events.sctp_association_event = 1;
events.sctp_address_event = 1;
events.sctp_send_failure_event = 1;
events.sctp_peer_error_event = 1;
events.sctp_adaption_layer_event = 1;
events.sctp_shutdown_event = 1;
events.sctp_partial_delivery_event = 1;

setsockopt(sd, IPPROTO_SCTP, SCTP_EVENTS, &events, sizeof(events));
```

Closing the association

A simple *close()* method invocation will initiate a graceful shutdown sequence (the SHUTDOWN chunk). But in case we need to close the association ungracefully (i.e., send an ABORT, instead of a SHUTDOWN chunk), we need to set the SO_LINGER socket option using the *setsockopt* method. If

this is set, any invocation of *close()* on the socket will initiate an ABORT chunk to the peer. For example:

```
struct linger ling={1,0};
setsockopt (sd, SOL_SOCKET, SO_LINGER, &ling, sizeof(ling));
```

The final picture

Let's use a few of the above methods and write a simple client-server program. The client sends a message to the server application that is echoed back by the server.

The following is the code for *server.c*:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sctp.h>

/* Method to handle SCTP Events */
void handle_event(void *buf)
{
    struct sctp_assoc_change *assoc_change;
    struct sctp_send_failed *send_failed;
    struct sctp_paddr_change *paddr_change;
    struct sctp_remote_error *remote_error;
    union sctp_notification *notif;

    notif = (sctp_notification *)buf;

    switch (notif->sn_header.sn_type)
    {
        case SCTP_ASSOC_CHANGE:
            assoc_change = &notif->sn_assoc_change;
            printf("Assoc Change: state: %u, error: %u, instreams: %u =\n",
                "outstreams: %u\n", assoc_change->sac_state, assoc_change->sac_error,
                assoc_change->sac_inbound_streams, assoc_change->sac_outbound_streams);
            break;

        case SCTP_SEND_FAILED:
            send_failed = &notif->sn_send_failed;
            printf("Send Failed: len: %u err: %u\n", send_failed->ssf_length,
                send_failed->ssf_error);
            break;

        case SCTP_PEER_ADDR_CHANGE:
            paddr_change = &notif->sn_paddr_change;
            if (paddr_change->spea_addr.ss_family == AF_INET)
            {
                struct sockaddr_in *sin;

```